qsBye

折腾笔记[28]-使用deepseek函数调用(FnCalling)进行自动化

摘要

使用ollama调用minicpm-v模型进行视觉场景认知, 然后将结果传递到deepseek进行移动决策和函数调用, 实现模拟控制机器人在场景中移动.

关键词

ollama;deepseek;minicpm;function calling;VLM;decision;

关键信息

- ollama version is 0.6.5
- MFDoom/deepseek-r1-tool-calling:1.5b 1.1 GB
- minicpm-v:latest 5.5 GB

原理简介

ollama保持模型在内存

[https://blog.csdn.net/canduecho/article/details/140647634]
Ollama 默认情况下，模型会在内存中保存 5 分钟，然后才会卸载。如果您向 LLM 发出大量请求，这可以缩短响应时间。但是，您可能希望在 5 分钟过去之前释放内存，或者无限期地加载模型。使用参数keep_alive和/api/generateAPI/api/chat端点来控制模型在内存中保留的时间。
该keep_alive参数可以设置为：

持续时间字符串（例如“10m”或“24h”）
秒数（例如 3600）
任何负数都会使模型保持在内存中（例如 -1 或“-1m”）
“0”将在生成响应后立即卸载模型
例如，要预加载模型并将其保留在内存中，请使用：

```
curl http://localhost:11434/api/generate -d '{"model": "llama2", "keep_alive": -1}'
```

ollama各种模型类别简介

[https://ollama.com/search]
[https://ollama.org.cn/blog/embedding-models]

- embedding
- vision
- tool

Embedding 模型

Embedding 模型主要用于将文本转换为高维向量（Embedding），这些向量可以用于语义理解、相似度计算、信息检索等任务。这些模型通常基于 Transformer 编码器（如 BERT、RoBERTa）进行构建，通过对比学习、掩码语言模型（MLM）等方式捕捉语义关系。

核心功能：

- 将文本转化为固定长度的数值向量，代表文本的语义特征。
- 用于 RAG（检索增强生成）、文本分类、知识库构建等应用。

技术架构与训练目标：

- 架构：基于 Transformer 编码器。
- 训练目标：优化文本特征提取能力。

典型使用场景：

- 信息检索：通过比较向量嵌入来搜索语义上相似的数据。
- 文本分类：利用文本的向量表示进行分类任务。
- 知识库构建：将文本信息存储在向量数据库中，用于快速检索。

性能与资源需求：

- 轻量化设计，部分模型仅需 400MB 显存（如 dmeta-embedding-zh）。
- 适合 CPU 推理，延迟通常在秒级。

Vision 模型

Vision 模型专注于处理图像相关任务，如图像分类、目标检测、图像生成等。这些模型可能采用卷积神经网络（CNN）、Vision Transformer（ViT）或扩散模型（Diffusion）。

核心功能：

- 处理图像相关任务，输出可能为图像标签、描述文本或生成的新图像。
- 用于多模态交互、图像内容理解、视觉问答（VQA）等应用。

技术架构与训练目标：

- 架构：可能采用 CNN、ViT 或扩散模型。
- 训练目标：依赖图像-文本配对数据（如 CLIP），或纯图像数据（如 ResNet）。

典型使用场景：

- 图像分类：识别图像中的对象或场景。
- 目标检测：在图像中定位和识别多个对象。
- 图像生成：基于文本描述生成新图像。

性能与资源需求：

- 资源消耗较高，尤其是高分辨率图像生成任务需 GPU 加速。
- 模型参数量较大（如 LLaVA 等多模态模型），显存需求可能超过 10GB。

Tool 模型

Tool 模型支持模型调用外部工具，使模型能够执行更复杂的任务或与外部世界交互。这些工具可以是函数、API、网页浏览、代码解释器等。

核心功能：

- 使模型能够通过调用外部工具来回答问题或执行任务。
- 支持更复杂的任务执行和外部世界交互。

技术架构与训练目标：

- 架构：基于现有的大型语言模型，增加工具调用功能。
- 训练目标：优化模型以理解和使用各种工具。

典型使用场景：

- 自动化任务执行：通过调用 API 完成特定任务。
- 信息检索：使用网页浏览工具获取最新信息。
- 代码执行：利用代码解释器工具运行和测试代码。

minicpm-v视觉语言大模型(VLM)简介

[https://sth.ai/article/MiniCPMV]
[https://openbmb.vercel.app/minicpm-v-2-en]
[https://github.com/OpenBMB/MiniCPM-V]
[https://openbmb.vercel.app/minicpm-v-2]
[https://zhuanlan.zhihu.com/p/713931969]
[https://arxiv.org/pdf/2408.01800]
[https://blog.csdn.net/HUSTHY/article/details/140873090]
MiniCPM-V是一系列专为视觉语言理解而设计的端侧多模态 LLM（MLLM）。该模型以图像和文本作为输入，并提供高质量的文本输出。自 2024 年 2 月以来，我们已发布了 4 个版本的模型，旨在实现强大的性能和高效的部署。该系列中目前最值得关注的模型包括：
MiniCPM-Llama3-V 2.5 :🔥🔥🔥 MiniCPM-V 系列中最新、最强大的模型，共 8B 参数，整体性能超越 GPT-4V-1106、Gemini Pro、Qwen-VL-Max、Claude 3 等自研模型，增强 OCR 和指令跟踪能力，支持英、中、法、西、德等30 余种语言的多模态对话。借助量化、编译优化以及 CPU 和 NPU 上的多种高效推理技术，MiniCPM-Llama3-V 2.5 可高效部署在端侧设备上。
MiniCPM-V 2.0：MiniCPM-V 系列中最轻量级的型号，2B 参数，整体性能超越 Yi-VL 34B、CogVLM-Chat 17B、Qwen-VL-Chat 10B 等较大型号，可以接受任意长宽比、最大 180 万像素（如 1344x1344）的图像输入，在场景文本理解方面达到与 Gemini Pro 相当的性能，在低幻觉率方面与 GPT-4V 相当。

该模型由三个关键模块组成:视觉编码器、压缩层和LLM。输入图像首先由视觉编码器编码,采用自适应视觉编码方法。具体来说,我们使用SigLIP SoViT-400m/14[115]作为视觉编码器。然后,视觉标记通过压缩层进行压缩,压缩层采用具有一层交叉注意力的感知器重采样器结构。最后,压缩后的视觉标记与文本输入一起输入LLM进行条件文本生成。

图像分割。为了处理具有不同纵横比的高分辨率图像,我们将图像分割成切片,每个切片更好地匹配ViT的预训练设置的分辨率和纵横比。具体来说,我们首先根据输入图像的大小计算理想切片数。给定一个分辨率为(WI,HI)的图像,和一个在分辨率为(Wv,Hv)的图像上预训练的ViT,我们计算理想切片数N = ⌈WI×HI / (Wv×Hv)⌉。然后,我们从集合CN = {(m, n)|m × n = N, m ∈ N, n ∈ N}中选择行数n和列数m的组合。一个好的分割(m, n)应该产生与ViT的预训练设置很好地匹配的切片。为了实现这一点,我们使用一个评分函数来评估每种可能的分割:S(m, n) = −∑(log(WI/m) + log(HI/n) − log(Wv/Hv))。

把原始输入tokenize后得到的input_ids,从中decode得到img_path，读取图片，经过resize、conv2d to patches一系列操作，把img tokenize化，可以输入到visionTransformer预训练模型中提取img_feature，最后经过adpater中的线性层把维度和文本特征维度对齐，才通过crossattention把img_feature压缩到固定token数量上，减少img特征对token的占用。

量化。量化是一种广泛使用的减少内存消耗的技术。模型量化的主要思想是使用统一的缩放因子将多个权重压缩到较窄的范围内,然后进行离散化。这个过程在数学上表示为:w′i = round(wi / s), ∀1 ≤ i ≤ n,

对于MiniCPM-Llama3-V 2.5,fp16版本的模型通常需要16到17GB的内存。我们选择GGML4框架中的Q4_K_M模式4位量化策略。这将内存需求降低到大约5GB,这对移动电话使用是友好的。

deepseek-function-calling函数调用大模型简介

[https://zhuanlan.zhihu.com/p/714036478]
[https://ollama.com/blog/functions-as-tools]
[https://blog.arunangshudas.com/implement-function-calling-for-tiny-llama/]
[https://ollama.com/blog/tool-support]
Function Call，或者叫函数调用、工具调用，是大语言模型中比较重要的一项能力，对于扩展大语言模型的能力，或者构建AI Agent，至关重要。

Ollama now supports tool calling with popular models such as Llama 3.1. This enables a model to answer a given prompt using tool(s) it knows about, making it possible for models to perform more complex tasks or interact with the outside world.

Example tools include:

Functions and APIs
Web browsing
Code interpreter
much more!

To enable tool calling, provide a list of available tools via the tools field in Ollama's API.

```
import ollama

response = ollama.chat(
    model='llama3.1',
    messages=[{'role': 'user', 'content':
        'What is the weather in Toronto?'}],

        # provide a weather checking tool to the model
    tools=[{
      'type': 'function',
      'function': {
        'name': 'get_current_weather',
        'description': 'Get the current weather for a city',
        'parameters': {
          'type': 'object',
          'properties': {
            'city': {
              'type': 'string',
              'description': 'The name of the city',
            },
          },
          'required': ['city'],
        },
      },
    },
  ],
)

print(response['message']['tool_calls'])
```

Supported models will now answer with a tool_calls response. Tool responses can be provided via messages with the tool role. See API documentation for more information.
请求:

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "What is the weather today in Paris?"
    }
  ],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_current_weather",
        "description": "Get the current weather for a location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "The location to get the weather for, e.g. San Francisco, CA"
            },
            "format": {
              "type": "string",
              "description": "The format to return the weather in, e.g. 'celsius' or 'fahrenhe
              "enum": ["celsius", "fahrenheit"]
            }
          },
          "required": ["location", "format"]
        }
      }
    }
  ]
}'
```

响应:

```
{
  "model": "llama3.2",
  "created_at": "2024-07-22T20:33:28.123648Z",
  "message": {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_current_weather",
          "arguments": {
            "format": "celsius",
            "location": "Paris, FR"
          }
        }
      }
    ]
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 885095291,
  "load_duration": 3753500,
  "prompt_eval_count": 122,
  "prompt_eval_duration": 328493000,
  "eval_count": 33,
  "eval_duration": 552222000
}
```

实现

1. uv新建工程

```
uv init
uv add ollama
uv add typing-extensions
uv run ollama_decision.py >> ../result/ollama_decision.log
```

工程配置:

```
[project]
name = "seekslam-examples"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "ollama>=0.4.8",
    "typing-extensions>=4.13.2",
]
```

下载模型:国内网速慢->获取ollama模型的下载链接
[https://www.reddit.com/r/ollama/comments/1eysgc1/get_direct_download_links_for_ollama_models_with/]
[https://www.reddit.com/r/ollama/comments/1dy8pqw/generating_modelfiles_from_huggingface/]
[https://github.com/amirrezaDev1378/ollama-model-direct-download]

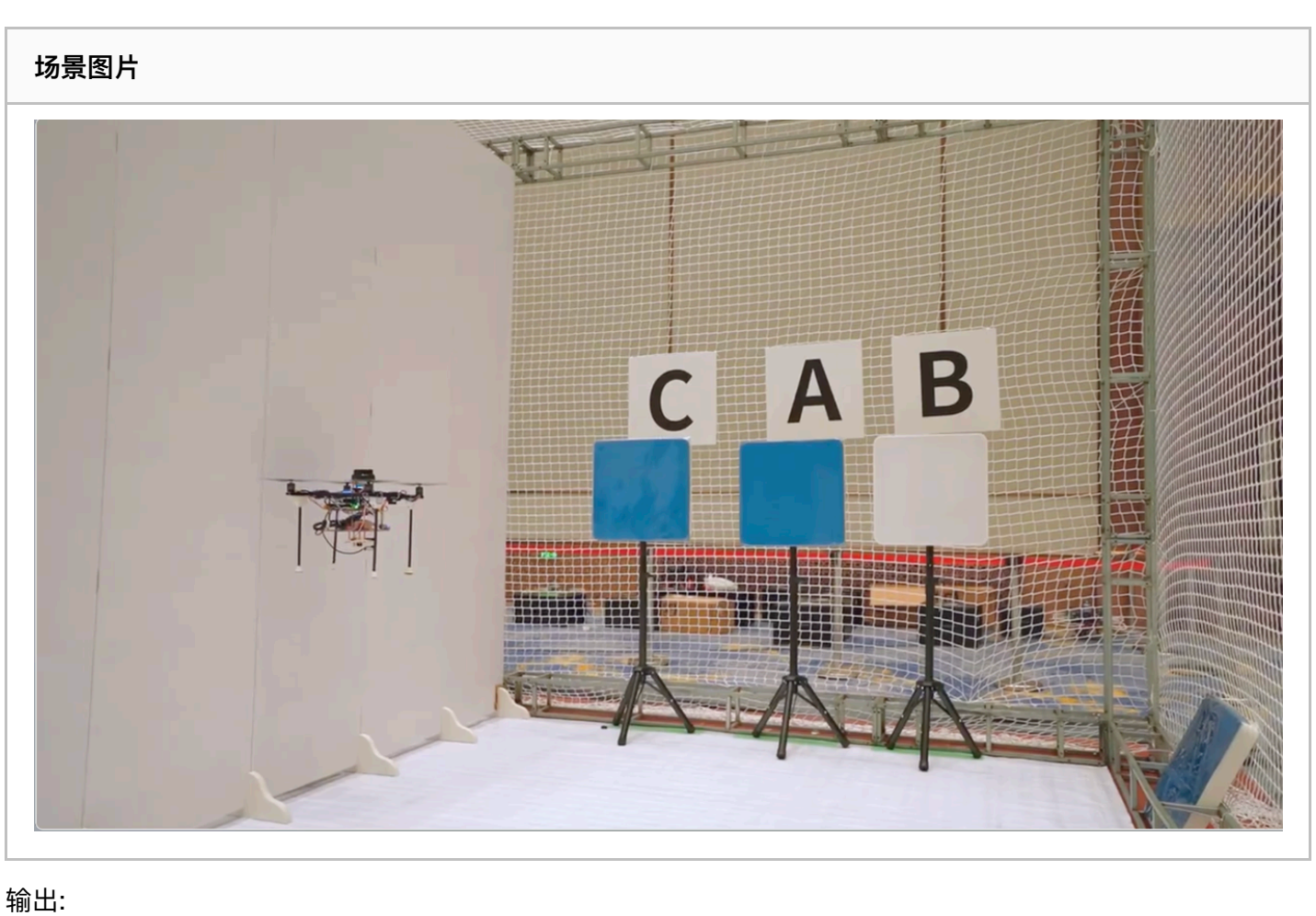
```
ollama pull MFDoom/deepseek-r1-tool-calling:1.5b
ollama pull minicpm-v:latest
```

2. 代码

```
# 无人机视觉识别与自主决策控制系统
# 功能：通过视觉大模型识别场景，然后使用函数调用模型进行决策和控制

import ollama
from ollama import Client
from typing_extensions import MutableMapping
import time
import json
import requests  # 用于直接调用API接口

# 系统常量定义
PROMPT_VISION = '图片里面有几个字母？描述字母相对无人机的位置关系，无人机更靠近哪个字母?'  # 视觉识别提示词
PROMPT_DECISION = '攻击目标为B，根据相对关系移动无人机到B前方'  # 决策提示词
MODEL_VISION = 'minicpm-v:latest'  # 视觉大模型
MODEL_TOOL = 'MFDoom/deepseek-r1-tool-calling:1.5b'  # 函数调用模型

# 初始化Ollama客户端
client = Client(host='http://localhost:11434')

def preload_models():
    """
    预加载模型到内存中并保持常驻
    """
    print("\n[系统] 开始预加载模型并保持常驻内存...")

    try:
        # 预加载视觉模型并保持常驻
        print("[系统] 正在预加载视觉模型并保持常驻...")
        response = requests.post(
            'http://localhost:11434/api/generate',
            json={
                'model': MODEL_VISION,
                'prompt': ' ',  # 空提示
                'keep_alive': -1  # 保持模型常驻内存
            }
        )
        response.raise_for_status()

        # 预加载函数调用模型并保持常驻
        print("[系统] 正在预加载函数调用模型并保持常驻...")
        response = requests.post(
            'http://localhost:11434/api/generate',
            json={
                'model': MODEL_TOOL,
                'prompt': ' ',  # 空提示
                'keep_alive': -1  # 保持模型常驻内存
            }
        )
        response.raise_for_status()

        print("[系统] 模型预加载完成并保持常驻内存")
    except Exception as e:
        print(f"[错误] 模型预加载失败: {str(e)}")
        raise

def check_model_status():
    """
    检查模型是否已加载
    """
    try:
        response = requests.get('http://localhost:11434/api/tags')
        response.raise_for_status()
        models = [model['name'] for model in response.json()['models']]

        print("\n[系统] 当前已加载模型:")
        for model in models:
            print(f" - {model}")

        return MODEL_VISION in models and MODEL_TOOL in models
    except Exception as e:
        print(f"[错误] 检查模型状态失败: {str(e)}")
        return False

def vision_recognition(image_path):
    """
    视觉识别处理函数
    参数:
        image_path: 图片文件路径
    返回:
        str: 视觉模型返回的场景描述
    """
    print("\n[系统] 开始视觉识别处理...")

    try:
        # 读取图片文件
        with open(image_path, 'rb') as f:
            image_data = f.read()

            # 调用视觉模型进行识别，保持模型常驻
            stream = client.generate(
                model=MODEL_VISION,
                prompt=PROMPT_VISION,
                images=[image_data],
                stream=True,
                keep_alive=-1  # 保持模型常驻内存
            )

            # 收集和处理响应
            full_response = []
            print("[系统] 已提交视觉识别请求，等待响应...")

            for chunk in stream:
                print(chunk['response'], end='', flush=True)
                full_response.append(chunk['response'])

            # 返回完整的场景描述
            return ''.join(full_response)
    except Exception as e:
        print(f"[错误] 视觉识别失败: {str(e)}")
        raise

def decision_making(scene_description):
    """
    决策和函数调用处理
    参数:
        scene_description: 视觉模型返回的场景描述
    """
    print("\n[系统] 开始决策和函数调用处理...")

    try:
        # 准备对话消息
        messages = [
            {'role': 'system', 'content': '你是一个无人机控制系统，需要根据视觉描述做出决策'},
            {'role': 'user', 'content': scene_description},
            {'role': 'user', 'content': PROMPT_DECISION}
        ]

        # 调用函数模型，保持模型常驻
        response = client.chat(
            model=MODEL_TOOL,
            messages=messages,
            tools=[move],  # 注册可调用的函数
            stream=True,
            keep_alive=-1  # 保持模型常驻内存
        )

        # 处理响应
        full_response = []
        tool_calls = []
        print("[系统] 已提交决策请求，等待响应...")

        for chunk in response:
            if 'message' in chunk:
                message = chunk['message']

                # 收集文本响应
                if 'content' in message and message['content']:
                    content = message['content']
                    print(content, end='', flush=True)
                    full_response.append(content)

                # 收集函数调用请求
                if 'tool_calls' in message:
                    tool_calls.extend(message['tool_calls'])

        # 处理函数调用
        if tool_calls:
            print("\n[系统] 检测到函数调用请求")
            for tool_call in tool_calls:
                func_name = tool_call['function']['name']
                args = tool_call['function']['arguments']

                print(f"[系统] 调用函数: {func_name}, 参数: {args}")

                # 执行对应的函数
                if func_name == "move":
                    try:
                        # 参数已经是字典格式，直接处理
                        if isinstance(args, str):
                            # 如果是字符串，先转换为字典
                            args = json.loads(args.replace("'", '"'))

                        # 确保所有值为数值类型
                        numeric_args = {
                            'duration': float(args.get('duration', 10)),
                            'forward': float(args.get('forward', 0)),
                            'backward': float(args.get('backward', 0)),
                            'left': float(args.get('left', 0)),
                            'right': float(args.get('right', 0)),
                            'up': float(args.get('up', 0)),
                            'down': float(args.get('down', 0))
                        }
                        move(**numeric_args)
                    except Exception as e:
                        print(f"[错误] 函数调用参数处理失败: {str(e)}")
                        raise

        # 记录完整日志
        with open('../result/ollama_decision.log', 'a', encoding='utf-8') as log_file:
            log_file.write(''.join(full_response))
            log_file.write('\n')
    except Exception as e:
        print(f"[错误] 决策处理失败: {str(e)}")
        raise

def move(duration=10.0, forward=0.0, backward=0.0, left=0.0, right=0.0, up=0.0, down=0.0):
    """
    无人机运动控制函数
    参数:
        duration: 运动持续时间(秒) 默认10秒
        forward: 向前速度(0~1) 默认0
        backward: 向后速度(0~1) 默认0
        left: 向左速度(0~1) 默认0
        right: 向右速度(0~1) 默认0
        up: 向上速度(0~1) 默认0
        down: 向下速度(0~1) 默认0
    """
    try:
        # 验证参数范围
        duration = max(0.1, min(float(duration), 60))  # 限制在0.1-60秒之间
        forward = max(0, min(float(forward), 1))
        backward = max(0, min(float(backward), 1))
        left = max(0, min(float(left), 1))
        right = max(0, min(float(right), 1))
        up = max(0, min(float(up), 1))
        down = max(0, min(float(down), 1))

        # 计算各轴速度
        velocity_x = forward - backward
        velocity_y = right - left
        velocity_z = down - up

        # 打印运动指令
        print(f"\n[控制] 执行运动指令:")
        print(f"时长: {duration}秒")
        print(f"X轴速度: {velocity_x:.2f} (前+/后-)")
        print(f"Y轴速度: {velocity_y:.2f} (右+/左-)")
        print(f"Z轴速度: {velocity_z:.2f} (下+/上-)")

    except Exception as e:
        print(f"[错误] 运动控制参数无效: {str(e)}")
        raise

# 主程序
if __name__ == "__main__":
    try:
        # 0. 预加载模型并保持常驻
        preload_models()

        # 检查模型状态
        if not check_model_status():
            print("[警告] 部分模型未正确加载，尝试重新加载...")
            preload_models()

        # 1. 视觉识别阶段
        image_path = '../assets/scene1.png'  # 图片路径
        scene_description = vision_recognition(image_path)

        # 2. 决策和控制阶段
        decision_making(scene_description)

        print("\n[系统] 任务执行完成")
    except Exception as e:
        print(f"[系统] 任务执行失败: {str(e)}")
```

效果

场景图片



输出:

```
[系统] 开始预加载模型并保持常驻内存...
[系统] 正在预加载视觉模型并保持常驻...
[系统] 正在预加载函数调用模型并保持常驻...
[系统] 模型预加载完成并保持常驻内存

[系统] 当前已加载模型:
 - MFDoom/deepseek-r1-tool-calling:1.5b
 - minicpm-v:latest
 - aleSuglia/qwen2-vl-2b-instruct-q4_k_m:latest
 - qllama/bge-reranker-large:latest
 - bge-m3:latest
 - qwen2.5:0.5b
 - deepseek-r1:latest
 - minicpm2b:latest

[系统] 开始视觉识别处理...
[系统] 已提交视觉识别请求，等待响应...
在提供的图片中，总共有三个大写字母可见：一个黑色的'C'和两个蓝色的'A'。此外，还有'B'字样的部分显示在一个板上。

至于无人机与字母的关系：

- 无人机悬挂在右侧墙壁上的一个支架上。
- 飞行器更靠近"A"标牌，因为它直接位于无人机正下方，并且是其飞行路径的最接近目标之一。

没有关于无人机在不同时间点位置或可能轨迹的具体信息，我们无法确定它是否已经更靠近"B"字母或者"C"字母。但是根据图

[系统] 开始决策和函数调用处理...
[系统] 已提交决策请求，等待响应...

[系统] 检测到函数调用请求
[系统] 调用函数: move, 参数: {'duration': 5}

[控制] 执行运动指令:
时长: 5.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: 0.00 (下+/上-)
[系统] 调用函数: move, 参数: {'forward': -1}

[控制] 执行运动指令:
时长: 10.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: 0.00 (下+/上-)
[系统] 调用函数: move, 参数: {'up': 0.5}

[控制] 执行运动指令:
时长: 10.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: -0.50 (下+/上-)
[系统] 调用函数: move, 参数: {'duration': '5'}

[控制] 执行运动指令:
时长: 5.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: 0.00 (下+/上-)
[系统] 调用函数: move, 参数: {'forward': '-1'}

[控制] 执行运动指令:
时长: 10.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: 0.00 (下+/上-)
[系统] 调用函数: move, 参数: {'up': '0.5'}

[控制] 执行运动指令:
时长: 10.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: -0.50 (下+/上-)

[系统] 任务执行完成

[系统] 开始预加载模型并保持常驻内存...
[系统] 正在预加载视觉模型并保持常驻...
[系统] 正在预加载函数调用模型并保持常驻...
[系统] 模型预加载完成并保持常驻内存

[系统] 当前已加载模型:
 - MFDoom/deepseek-r1-tool-calling:1.5b
 - minicpm-v:latest
 - aleSuglia/qwen2-vl-2b-instruct-q4_k_m:latest
 - qllama/bge-reranker-large:latest
 - bge-m3:latest
 - qwen2.5:0.5b
 - deepseek-r1:latest
 - minicpm2b:latest

[系统] 开始视觉识别处理...
[系统] 已提交视觉识别请求，等待响应...
在提供的图片中，可以看到有四个大写字母：'C', 'A', 'B', 和一个额外的蓝色正方形。这些字母是放在支架上的，并摆放

要确定无人机更接近于哪一个字母，请注意以下几点：

1. **无人机的位置**:
   - 无人机悬停在图片左侧，略微偏上。

2. **字母的距离和方向**:
   - 'C' 和 'A' 分别位于最左边的两个支架上方，并且直接相邻。它们的高度接近于同一水平线。

3. **额外蓝色正方形的位置**:
   - 在右侧的支架旁边是一个空白的蓝色方块，它在 'B' 旁边的支架上，因此它是与 'C', 'A', 和 'B' 最靠近的物体

由于无人机悬停在图片左侧且略微偏上，并考虑到字母和额外蓝色正方形的位置关系，可以合理地推测无人机更接近于左边。然

因此，图片中无人机位于左侧，并且看起来它离左边两个支架上的'CA'标牌较近，而不是右侧的蓝色正方形和'B'标牌。

[系统] 开始决策和函数调用处理...
[系统] 已提交决策请求，等待响应...

[系统] 检测到函数调用请求
[系统] 调用函数: move, 参数: {'duration': 5}

[控制] 执行运动指令:
时长: 5.0秒
X轴速度: 0.00 (前+/后-)
Y轴速度: 0.00 (右+/左-)
Z轴速度: 0.00 (下+/上-)

[系统] 任务执行完成
```

分类: 人工智能 , 折腾笔记
标签: LLM

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 折腾笔记[27]-使用OCR检测和识别字母

posted @ 2025-04-19 04:34 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

公告
昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0

2025年4月

日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

搜索

常用链接
我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签
Rust(19)
SLAM(8)
LLM(2)
线性代数(2)

随笔分类
51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(6)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(13)
前端开发(18)
嵌入式Linux(3)
人工智能(5)
运维相关(16)
折腾笔记(30)

随笔档案
2025年4月(12)
2025年3月(4)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
更多

阅读排行榜
1. OpenWrt安装Filebrowser网页文件管理器(11832)
2. esp32笔记[5]-基于I2S协议实现音频播放(9809)
3. 谈谈NAS的硬盘SATA背板(6875)
4. 三星平板的笔记模板(比例:16:10)(4375)
5. 电犀牛R66s软路由体验(4329)

评论排行榜
1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜
1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[1]-blink闪灯(1)
4. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

最新评论
1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】博客园2025新款「AI繁忙」系列T恤上架，前往周边小店选购
【推荐】凌霞软件回馈社区，携手博客园推出1Panel与Halo联合会员
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：

- 解锁.NET 9性能优化黑科技：从内存管理到Web性能的最全指南
- 通过一个DEMO理解MCP（模型上下文协议）的生命周期
- MySQL下200GB大表备份，利用传输表空间解决停服发版表备份问题
- 记一次 .NET某固高运动卡测试 卡慢分析
- 微服务架构学习与思考：微服务拆分的原则

阅读排行：

- 解锁.NET 9性能优化黑科技：从内存管理到Web性能的最全指南
- .net clr 8年才修复的BUG，你让我损失太多了
- 一个神奇的JS代码，让浏览器在新的空白标签页运行我们 HTML 代码（createObjectURL
- 即时通信SSE和WebSocket对比
- 做Docx预览，一定要做这个神库！！

历史上的今天：

2024-04-19 ROS笔记[2]-获取OpenMV数据并发布到ROS消息

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes